字符串 String/

> 依赖 `Template/ihxnan.hpp` 的类型别名（`ll`、`ul`、`vi`、`vvi` 等）。**索引约定不一**：`KMP.hpp` 三个函数均为 **1-indexed**（`str[0]` 为占位符）；其余（`StrHash`、`SqAM`、`SuffixArray`）为 **0-indexed**；`Trie` / `AhoCorasick` 为节点编号约定，见各自说明。

KMP / Z 函数（1-indexed）`KMP.hpp`

- **用法**：传入的 `str` 需为 1-indexed（`str[0]` 占位，实际字符在 `1..n`，如 `"$[文本]"` 风格构造）。
  - `get_fail(str)`：返回 `fail[1..n]`，`fail[i]` 为 `str[1..i]` 的最长相等真前后缀长度（`fail[0]` 为 `0`）。
  - `get_z(str)`：Z 函数，`z[1] = n`，`z[i]` 为 `str` 与 `str[i..]` 的 LCP 长度。
  - `get_tr(str, fail)`：KMP 自动机转移表，`tr[i][c]` 为位置 `i` 后接字符 `c` 时的转移目标。
- **复杂度**：均为 `O(n)`（`get_tr` 乘上字符集 26）。

```
vector<int> get_fail(const string &str) // KMP 失配数组
{
    int n = str.size() - 1; // str[0] 为占位符
    vector<int> fail(n + 1);
    for (int i = 2, j = 0; i ⩽ n; ++i)
    {
        while (j && str[i] ≠ str[j + 1]) // 失配沿 fail 回跳
            j = fail[j];
        if (str[i] == str[j + 1])
            ++j;
        fail[i] = j;
    }
    return fail;
}

vector<vector<int>> get_tr(const string &str, const vector<int> &fail) // KMP 自动机
{
    int n = fail.size() - 1;
    vector<vector<int>> tr(fail.size(), vector<int>(26));
    for (int i = 0; i < n; ++i)
    {
        for (int j = 0; j < 26; ++j)
            tr[i][j] = tr[fail[i]][j];  // 继承 fail 的转移
        tr[i][str[i + 1] - 'a'] = i;    // 匹配字符的转移
    }
    return tr;
}

vector<int> get_z(const string &str) // Z 函数
{
    int n = str.size() - 1; // str[0] 为占位符
    vector<int> z(n + 1);
    if (n == 0)
        return z;
    z[1] = n;
    for (int i = 2, l = 0, r = 0; i ⩽ n; i++)
    {
        if (i ⩽ r)
            z[i] = min(r - i + 1, z[i - l + 1]); // 利用窗口 [l, r] 信息
        while (i + z[i] ⩽ n && str[1 + z[i]] == str[i + z[i]])
            z[i]++; // 暴力扩展
        if (i + z[i] - 1 > r)
            l = i, r = i + z[i] - 1; // 更新右端最远窗口
    }
    return z;
}
```

字符串哈希（自然溢出，0-indexed） StrHash.hpp

- **用法**：StrHash hs(s);（下标 0..n-1，区间均为闭区间 [l, r]）；get(l, r) 正向哈希、rget(l, r) 反向哈希、is_palindrome(l, r) 回文判断、same(l1, r1, l2, r2) 子串相等、link(l1, r1, l2, r2) 拼接两段哈希、full() 整串哈希。
- **实现**：底数 13331，**自然溢出**（ul = unsigned long long，模 2^64）。
- **注意**：自然溢出有被构造数据卡的风险，赛时可把 base 换成随机数。

```
struct StrHash { // 字符串哈希（自然溢出，0-indexed，区间闭 [l, r]）
    ul base;               // 底数
    vector<ul> pow, h, rh; // pow: 幂次；h: 正向前缀哈希；rh: 反向后缀哈希

    StrHash(const string &str) : base(13331)
    {
        int n = str.size();
        pow.resize(n + 1);
        h.resize(n + 1);
        rh.resize(n + 1);
        pow[0] = 1, h[0] = 0, rh[n] = 0;
        for (int i = 0; i < n; i++) // 正向：h[i+1] = s[0..i] 的哈希
        {
            pow[i + 1] = pow[i] * base;
            h[i + 1] = h[i] * base + str[i];
        }
        for (int i = n - 1; i >= 0; i--) // 反向：rh[i] = s[i..n-1] 的哈希
            rh[i] = rh[i + 1] * base + str[i];
    }

    ul full() // 整串哈希
    {
        return h.back();
    }

    ul get(int l, int r) // 子串 s[l..r] 正向哈希
    {
        return h[r + 1] - h[l] * pow[r - l + 1];
    }

    ul rget(int l, int r) // 子串 s[l..r] 反向哈希
    {
        return rh[l] - rh[r + 1] * pow[r - l + 1];
    }

    ul link(int l1, int r1, int l2, int r2) // 拼接 s[l1..r1] + s[l2..r2]
    {
        return get(l1, r1) * pow[r2 - l2 + 1] + get(l2, r2);
    }

    bool same(int l1, int r1, int l2, int r2) // 两子串是否相等
    {
        return get(l1, r1) == get(l2, r2);
    }

    bool is_palindrome(int l, int r) // s[l..r] 是否回文
    {
        return get(l, r) == rget(l, r);
    }
};
```

Trie（62 字符集，支持计数 / 删除） Trie.hpp

- **用法**：Trie tr(预计节点数);；insert(s) 插入（pass 沿途 +1，end 终点 +1）；search(s) 完整单词 s 的个数；query(s) 以 s 为前缀的单词个数；del(s) 删除一个 s（沿 pass 减、无经过即剪枝）；clear() 清空复用。
- **字符集**：a-z（0-25）、A-Z（26-51）、0-9（52-61），共 62。
- **注意**：构造参数为节点数上限（内部开 n+5 个节点），传"字符串总长"即可；节点 0 为根。

```
struct Trie // 字典树（62 字符集：a-z A-Z 0-9，节点 0 为根）
{
    int tot;                        // 当前节点计数
    vector<int> pass, end;          // pass: 经过该节点的单词数；end: 以该节点结尾的单词数
    vector<vector<int>> trie;       // trie[u][c]: u 的字符 c 儿子（0 表示无）

    int get(const char &x) // 字符映射到 0..61
    {
        if (x ⩾ 'a' && x ⩽ 'z')
            return x - 'a';
        if (x ⩾ 'A' && x ⩽ 'Z')
            return x - 'A' + 26;
        return x - '0' + 52;
    }

    Trie(int n) : tot(0), pass(n + 5), end(n + 5), trie(n + 5, vector<int>(62))
    {
    }

    void insert(const string &str)
    {
        int cur = 0;
        ++pass[cur]; // 根也计数
        for (auto &p : str)
        {
            int x = get(p);
            if (!trie[cur][x])
                trie[cur][x] = ++tot; // 无儿子则新建
            cur = trie[cur][x];
            ++pass[cur];
        }
        ++end[cur];
    }

    int search(const string &str) // 完整单词 str 的个数
    {
        int cur = 0;
        for (auto &p : str)
        {
            int x = get(p);
            if (!trie[cur][x])
                return 0;
            cur = trie[cur][x];
        }
        return end[cur];
    }

    int query(const string &str) // 以 str 为前缀的单词个数
    {
        int cur = 0;
        for (auto &p : str)
        {
            int x = get(p);
            if (!trie[cur][x])
                return 0;
```

```
            cur = trie[cur][x];
        }
        return pass[cur];
    }

    void del(const string &str) // 删除一个 str（需已插入）
    {
        if (search(str) > 0)
        {
            int cur = 0;
            --pass[cur];
            for (auto &p : str)
            {
                int x = get(p);
                if (!--pass[trie[cur][x]]) // 无经过：直接剪枝
                {
                    trie[cur][x] = 0;
                    return;
                }
                cur = trie[cur][x];
            }
            --end[cur];
        }
    }

    void clear() // 清空所有节点（可复用）
    {
        for (int i = 0; i ⩽ tot; ++i)
        {
            pass[i] = end[i] = 0;
            fill(trie[i].begin(), trie[i].end(), 0);
        }
        tot = 0;
    }
};
```

AC 自动机（小写字母）AhoCorasick.hpp

- **用法**：AhoCorasick ac;；对每个模式串 add(s)（返回末节点，可在此打标记）；全部加完后 work() BFS 建 fail 指针并补全转移；之后 next(p, c) 在自动机上转，link(p) 取 fail。
- **结构**：节点 0 为哨兵（next 全指向根，len = -1），节点 1 为根；len 为节点深度（对应模式串前缀长度）。
- **常用套路**：文本沿 next 转移；fail 树上子树求和 / 拓扑序 DP 统计各模式串出现次数。
- **注意**：仅支持小写字母（ALPHABET = 26）。

```
struct AhoCorasick
{
    static constexpr int ALPHABET = 26;
    struct Node
    {
        int len;                        // 节点深度（该前缀长度）
        int link;                       // fail 指针
        array<int, ALPHABET> next;      // 自动机转移（已补全）
        Node() : len{0}, link{0}, next{}
        {
        }
    };

    vector<Node> tree;

    AhoCorasick()
    {
        init();
    }

    void init() // 节点 0 哨兵 + 节点 1 根
    {
        tree.assign(2, Node());
        tree[0].next.fill(1); // 哨兵所有转移指向根
        tree[0].len = -1;
    }

    int newNode()
    {
        tree.emplace_back();
        return tree.size() - 1;
    }

    int add(const string &str) // 插入模式串，返回末节点
    {
        int p = 1; // 从根出发
        for (auto c : str)
        {
            int x = c - 'a';
            if (tree[p].next[x] == 0)
            {
                tree[p].next[x] = newNode();
                tree[tree[p].next[x]].len = tree[p].len + 1;
            }
            p = tree[p].next[x];
        }
        return p;
    }

    void work() // BFS 建 fail 指针，并把缺省转移补全
    {
        queue<int> que;
        que.push(1);
```

```
        while (!que.empty())
        {
            int x = que.front();
            que.pop();

            for (int i = 0; i < ALPHABET; i++)
            {
                if (tree[x].next[i] == 0)
                {
                    tree[x].next[i] = tree[tree[x].link].next[i]; // 补全转移
                }
                else
                {
                    tree[tree[x].next[i]].link = tree[tree[x].link].next[i]; // fail
                    que.push(tree[x].next[i]);
                }
            }
        }
    }

    int next(int p, int x) // p 状态遇字符 x 的转移
    {
        return tree[p].next[x];
    }

    int link(int p) // fail 指针
    {
        return tree[p].link;
    }

    int len(int p) // 节点深度
    {
        return tree[p].len;
    }

    int size() // 节点总数
    {
        return tree.size();
    }
};
```

子序列自动机 SequenceAM.hpp

- **用法**：SqAM sam(s);（0-indexed）；find(t) 判断 t 是否为 s 的**子序列**。
- **实现**：next[i][c] = 位置 i 及之后第一个字符 c 出现的下标（无则 -1）；匹配时 idx = next[idx][c] + 1。
- **复杂度**：预处理 O(26n) 时间与空间，匹配 O(|t|)。
- **注意**：内存开销 26n 个 int，n 很大时注意空间（可用 n log 26 的倍增 / 逐字符贪心替代）。

```
struct SqAM // 子序列自动机：0-indexed
{
    int n;                      // 串长
    vector<vector<int>> next; // next[i][c]: 位置 i 及之后首个字符 c 的下标（无则 -1）
    SqAM() = default;
    SqAM(const string &str)
    {
        init(str);
    }
    void clear()
    {
        n = 0, next.clear();
    }
    void init(const string &str) // 预处理：从右往左递推
    {
        n = str.size();
        next.resize(n + 1, vector<int>(26, -1));
        for (int i = n - 1; i >= 0; --i)
        {
            for (int j = 0; j < 26; ++j)
                next[i][j] = next[i + 1][j];   // 继承右侧
            next[i][str[i] - 'a'] = i;          // 覆盖当前字符
        }
    }
    bool find(const string &str) // str 是否为 s 的子序列
    {
        int idx = 0;
        for (auto p : str)
        {
            idx = next[idx][p - 'a'] + 1; // 跳到下一个匹配位置之后
            if (idx <= 0)                 // next 为 -1 时加 1 得 0：失配
                return false;
        }
        return true;
    }
};
```

后缀数组（倍增 + 计数排序，0-indexed） SuffixArray.hpp

- **用法**：SuffixArray sa(s);（s 为 0-indexed）；成员 sa[i]：按字典序第 i 小的后缀起点；rk[i]：后缀 i 的排名；lc[i]：lcp(sa[i], sa[i+1])（长度 n-1）。
- **复杂度**：O(n log n)。
- **注意**：n == 1 时 lc 为空；常见配合——任意两后缀 LCP 为 lc 数组上 min(rk[u] .. rk[v]-1)（RMQ/ST 表）。

```
struct SuffixArray // 后缀数组（倍增 + 计数排序）：0-indexed
{
    int n;                  // 串长
    vector<int> sa, rk, lc; // sa: 名次→后缀起点；rk: 后缀起点→名次；lc[i]=lcp(sa[i], sa[i+1])，长度 n
-1

    SuffixArray(const string &s)
    {
        n = s.length();
        sa.resize(n);
        lc.resize(n - 1);
        rk.resize(n);
        iota(sa.begin(), sa.end(), 0);
        sort(sa.begin(), sa.end(), [&](int a, int b) { return s[a] < s[b]; }); // 按首字符排序
        rk[sa[0]] = 0;
        for (int i = 1; i < n; i++) // 首字符排名
            rk[sa[i]] = rk[sa[i - 1]] + (s[sa[i]] != s[sa[i - 1]]);
        int k = 1;
        vector<int> tmp;
        vector<int> cnt(n);
        tmp.reserve(n);
        while (rk[sa[n - 1]] < n - 1) // 排名未全部分开，倍增
        {
            // 以长度 k 的排名为关键字，对长度 2k 排序
            // 1) 生成第二关键字顺序：越界后缀最小，其余为 sa[i]-k
            tmp.clear();
            for (int i = 0; i < k; i++)
                tmp.push_back(n - k + i);
            for (auto i : sa)
                if (i >= k)
                    tmp.push_back(i - k);
            // 2) 对第一关键字计数排序
            fill(cnt.begin(), cnt.end(), 0);
            for (int i = 0; i < n; i++)
                cnt[rk[i]]++;
            for (int i = 1; i < n; i++)
                cnt[i] += cnt[i - 1];
            for (int i = n - 1; i >= 0; i--)
                sa[--cnt[rk[tmp[i]]]] = tmp[i];
            // 3) 重算长度 2k 的排名
            swap(rk, tmp);
            rk[sa[0]] = 0;
            for (int i = 1; i < n; i++)
                rk[sa[i]] = rk[sa[i - 1]] +
                            (tmp[sa[i - 1]] < tmp[sa[i]] || sa[i - 1] + k == n || tmp[sa[i - 1] + k] < t
mp[sa[i] + k]);
            k *= 2;
        }
        // O(n) 求 height 数组 lc（利用 h[i] >= h[i-1]-1）
        for (int i = 0, j = 0; i < n; i++)
            if (rk[i] == 0)
                j = 0;
            else
            {
                for (j -= (j > 0); i + j < n && sa[rk[i] - 1] + j < n && s[i + j] == s[sa[rk[i] - 1] +
```

```
j]; j++)
                    ;
                lc[rk[i] - 1] = j;
            }
    }
};
```